



K.R. MANGALAM UNIVERSITY
THE COMPLETE WORLD OF EDUCATION

Fundamentals Of Operating
System Lab
(ENCA252)

Lab File

Submitted by: Greaty Raghav (2401201082)

Submitted to

Ms. Jyoti Yadav

School of Engineering & Technology

Topic: Implementation and Analysis of CPU Scheduling Algorithms (FCFS & SJF)

1. Aim

To implement First Come First Serve (FCFS) and Shortest Job First (SJF) scheduling algorithms and analyze their performance using parameters such as Completion Time (CT), Turnaround Time (TAT), and Waiting Time (WT).

2. Theory

CPU Scheduling is a process by which the operating system decides which process will execute next. It helps in efficient utilization of CPU.

FCFS (First Come First Serve):

- Processes are executed in the order of their arrival time.
- It is a non-preemptive scheduling algorithm.
- Simple but may lead to higher waiting time.

SJF (Shortest Job First):

- Process with the smallest burst time is executed first.
- It minimizes average waiting time.
- Also non-preemptive in this implementation.

3. Algorithm

FCFS Algorithm:

1. Sort processes based on arrival time.
2. Execute each process in order.
3. Calculate CT, $TAT = CT - AT$, $WT = TAT - BT$.

SJF Algorithm:

1. Select processes that have arrived.
2. Choose process with minimum burst time.
3. Execute and repeat until all processes complete.

4. Code

```
1 class Process:
2     def __init__(self, pid, at, bt):
3         self.pid = pid
4         self.at = at
5         self.bt = bt
6         self.ct = 0
7         self.tat = 0
8         self.wt = 0
9
10
11 def print_table(processes):
12     print("\nPID\tAT\tBT\tCT\tTAT\tWT")
13     for p in processes:
14         print(f"{p.pid}\t{p.at}\t{p.bt}\t{p.ct}\t{p.tat}\t{p.wt}")
15
16
17 # ===== FCFS =====
18 def fcfs(processes):
19     processes.sort(key=lambda x: x.at)
20     time = 0
21     gantt = []
22
23     for p in processes:
24         if time < p.at:
25             gantt.append(("Idle", time, p.at))
26             time = p.at
27
28         start = time
29         time += p.bt
30         p.ct = time
31         p.tat = p.ct - p.at
32         p.wt = p.tat - p.bt
33
34         gantt.append((p.pid, start, time))
35
36     print("\n--- FCFS ---")
37     print_table(processes)
38
39     avg_wt = sum(p.wt for p in processes) / len(processes)
40     avg_tat = sum(p.tat for p in processes) / len(processes)
41
42     print(f"\nAverage WT = {avg_wt:.2f}")
43     print(f"Average TAT = {avg_tat:.2f}")
44
45     print("\nGantt Chart:")
46     for g in gantt:
47         print(f"| {g[0]} |", end="")
48     print("|")
49
50     for g in gantt:
51         print(f"| {g[1]}\t", end="")
52     print(f"| {g[2]} |")
53
54
55 # ===== SJF =====
56 def sjf(processes):
57     processes = sorted(processes, key=lambda x: (x.at, x.bt))
58     completed = []
59     time = 0
60     ready = []
61     gantt = []
62
63     processes_copy = processes[:]
64
65     while processes_copy or ready:
66         while processes_copy and processes_copy[0].at <= time:
67             ready.append(processes_copy.pop(0))
68
69         if not ready:
70             time += 1
71             continue
72
73         ready.sort(key=lambda x: x.bt)
74         p = ready.pop(0)
75
76         start = time
77         time += p.bt
78
79         p.ct = time
80         p.tat = p.ct - p.at
81         p.wt = p.tat - p.bt
82
83         gantt.append((p.pid, start, time))
84         completed.append(p)
85
86     print("\n--- SJF (Non-Preemptive) ---")
87     print_table(completed)
88
89     avg_wt = sum(p.wt for p in completed) / len(completed)
90     avg_tat = sum(p.tat for p in completed) / len(completed)
91
92     print(f"\nAverage WT = {avg_wt:.2f}")
93     print(f"Average TAT = {avg_tat:.2f}")
94
95     print("\nGantt Chart:")
96     for g in gantt:
97         print(f"| {g[0]} |", end="")
98     print("|")
99
100     for g in gantt:
101         print(f"| {g[1]}\t", end="")
102     print(f"| {g[2]} |")
103
104
105 # ===== MAIN =====
106 n = int(input("Enter number of processes: "))
107 processes = []
108
109 for i in range(n):
110     at = int(input(f"Enter Arrival Time for P{i+1}: "))
111     bt = int(input(f"Enter Burst Time for P{i+1}: "))
112     processes.append(Process(f"P{i+1}", at, bt))
113
114 # Copy list for separate execution
115 import copy
116 fcfs(copy.deepcopy(processes))
117 sjf(copy.deepcopy(processes))
```

5. Input Used

Example Input:

Number of Processes = 5

| Process | AT | BT |

|-----|----|----

| P1 | 0 | 8 |

| P2 | 1 | 4 |

| P3 | 2 | 9 |

| P4 | 3 | 5 |

| P5 | 4 | 2 |

6. Output

```
Enter number of processes: 5
Enter Arrival Time for P1: 7
Enter Burst Time for P1: 8
Enter Arrival Time for P2: 4
Enter Burst Time for P2: 2
Enter Arrival Time for P3: 8
Enter Burst Time for P3: 6
Enter Arrival Time for P4: 5
Enter Burst Time for P4: 9
Enter Arrival Time for P5: 1
Enter Burst Time for P5: 2
```

--- FCFS ---

PID	AT	BT	CT	TAT	WT
P5	1	2	3	2	0
P2	4	2	6	2	0
P4	5	9	15	10	1
P1	7	8	23	16	8
P3	8	6	29	21	15

Average WT = 4.80
Average TAT = 10.20

Gantt Chart:

```
| Idle | P5 | Idle | P2 | P4 | P1 | P3 |
0      1      3      4      6      15      23      29
```

--- SJF (Non-Preemptive) ---

PID	AT	BT	CT	TAT	WT
P5	1	2	3	2	0
P2	4	2	6	2	0
P4	5	9	15	10	1
P3	8	6	21	13	7
P1	7	8	29	22	14

Average WT = 4.40
Average TAT = 9.80

Gantt Chart:

```
| P5 | P2 | P4 | P3 | P1 |
1      4      6      15      21      29
```

- FCFS Output Table
- SJF Output Table
- Gantt Charts

7. Result

- FCFS executes processes in arrival order, which may lead to higher waiting time.
- SJF selects the shortest job first, reducing the average waiting time.
- From the results, it is observed that SJF performs better than FCFS in terms of efficiency.

8. Conclusion

CPU scheduling plays an important role in process management.

Among the two algorithms implemented, SJF provides better performance by minimizing waiting time, while FCFS is simpler but less efficient.

9. GitHub Link

(<https://github.com/Greaty01-Work/os-lab-assignments.git>)

```

68         if not ready:
69             time += 1
70             continue
71
72         ready.sort(key=lambda x: x.bt)
73         p = ready.pop(0)
74
75         start = time
76         time += p.bt
77
78         p.ct = time
79         p.tat = p.ct - p.at
80         p.wt = p.tat - p.bt
81
82         gantt.append((p.pid, start, time))
83         completed.append(p)
84
85     print("\n--- SJF (Non-Preemptive) ---")
86     print_table(completed)
87
88     avg_wt = sum(p.wt for p in completed) / len(completed)
89     avg_tat = sum(p.tat for p in completed) / len(completed)
90
91     print(f"\nAverage WT = {avg_wt:.2f}")
92     print(f"Average TAT = {avg_tat:.2f}")
93
94     print("\nGantt Chart:")
95     for g in gantt:
96         print(f"[{g[0]} {g[1]} {g[2]} ", end="")
97     print("")
98
99     for g in gantt:
100         print(f"[{g[1]} {g[2]} ", end="")
101     print(f"[{g[0]} {g[1]} {g[2]}"]
102
103 # ===== MAIN =====
104 n = int(input("Enter number of processes: "))
105 processes = []
106
107 for i in range(n):
108     at = int(input(f"Enter Arrival Time for P{i+1}: "))
109     bt = int(input(f"Enter Burst Time for P{i+1}: "))
110     processes.append(Process(f"P{i+1}", at, bt))
111
112 # Copy list for separate execution
113 import copy
114 rcfs(copy.deepcopy(processes))
115 sjf(copy.deepcopy(processes))

```